

Artificial Intelligence Methods in Software Refactoring: A Systematic Literature Review

^{1st} Simona Motogna

*Faculty of Mathematics and Computer Science
Babes-Bolyai University
Cluj-Napoca, Romania
simona.motogna@ubbcluj.ro*

^{2nd} Liviu-Marian Berciu

*Faculty of Mathematics and Computer Science
Babes-Bolyai University
Cluj-Napoca, Romania
liviu.berciu@ubbcluj.ro*

^{3rd} Vasilica-Andreea Moldovan

*Faculty of Mathematics and Computer Science
Babes-Bolyai University
Cluj-Napoca, Romania
vasilica.moldovan@ubbcluj.ro*

Abstract—Refactoring is an important process in software engineering, aiming to improve code quality without altering the behavior. This article presents a systematic review of the literature (SLR) on artificial intelligence in the domain of software refactoring. Following a rigorous methodology consisting of data extraction, snowballing techniques, and manual validation, we created a dataset consisting of 156 articles. The focus of the investigation was to identify the refactoring stages that are addressed. The results show that, as research type, most of the contributions propose solutions, while other forms of research such as evaluation, validation and experience are less represented in publications. Refactoring detection represents the highest interest in research contributions, while other refactoring stages, such as prioritization or testing are less investigated. The most commonly used AI methods include Random Forests, Genetic Algorithms, SVM, CNNs and Decision Trees. Based on this literature review, we have identified research trends and opportunities for future research.

Index Terms—Refactoring, Artificial Intelligence, AI in Software Refactoring, Machine Learning, Systematic Literature Review

I. INTRODUCTION

Refactoring represents an important process in software development, aiming to improve code structure, maintainability and reusability, without modifying its functionality, as defined in [1]. It has represented a continuous interest from the research community due to its impact on code quality, performance or energy consumption [2]–[4].

The last years have experienced an increased interest in applying Artificial Intelligence (AI) and especially Machine Learning (ML) approaches to different software engineering problems. This research interest is due to rapid advancements in AI/ML algorithms, but also to the availability of large software related datasets. Examples include mining software repositories, code smell detection, test case generation or development effort [5], to name a few. An important boost to the domain is represented by the availability of dedicated tools, such as WEKA [6] or Scikit-Learn [7].

This also stands for approaches dealing with different aspects of refactoring. With our contribution, we hope to give a comprehensive overview of the interplay between refactoring and AI/ML techniques. Previous studies have tackled the domain of refactoring, but either as a whole [8] or from the perspective of quality attributes [9].

We formulate our main objectives in undertaking this study as a systematic literature review (SLR) that constructs a map of state-of-the-art application of AI/ML methods to refactoring stages.

The SLR was performed based on a defined protocol and the primary studies that were included were selected from scientific literature sources. We provide the created dataset for further use in the research community. The quantitative analysis conducted identifies trends and open subjects within the explored domain.

The rest of the paper is organized as follows: we start by presenting the applied SLR methodology. Section III reports the results of our systematic review, followed by discussions and implications. Section V investigates related work, while Section VI presents threats to validity. We end with some conclusions related to our findings.

II. RESEARCH METHODOLOGY

Our approach to literature review is based on [10], defining it as a scoping literature review with the purpose of describing the domain of AI/ML in relation with refactoring software systems and mapping the studies from this domain. We investigated existing standards in the domain [11] and similar studies in software engineering [8], [9], [12], [13]. We also followed the guidelines described in [12] and documented each step as summarized in Table I. Steps 1 and 2 belong to planning, steps 3 and 4 for conducting the review, and step 5 refers to reporting. This section describes details about each step.

TABLE I
PROTOCOL FOLLOWED FOR SLR

No.	Step	Description
1	Goal	Impact of AI/ML on refactoring
2	Research Questions	Which refactoring phases? Which AI/ML methods?
3	Search Strategy	Databases for primary studies What keywords will be used? What information is retrieved?
4	Selection criteria	Inclusion criteria Exclusion criteria
5	Synthesis of data	Information to be synthesized Data Analysis

A. Research Questions

The research objective of discussing the state-of-the-art in the field of applying AI/ML methods to refactoring lifecycle stages is structured into the following research questions:

RQ1: *What life-cycle phases of refactoring are addressed?* Refactoring is a complex process that incorporates several phases, starting from detection, prioritization, and recommendation, processes that should be accompanied by documentation and testing, and might be predicted. There are several classifications/namings of these phases (such as in [14] or [8]). We target to investigate how the selected studies target these phases.

RQ2: *Which AI/ML methods are used?* Our exploration will inquire what type of AI methods are used and which are the most frequent AI/ML applied for refactoring.

B. Search Strategy

The search strategy steps are detailed in Figure 1.



Fig. 1. Steps followed in SLR

We started by establishing the scientific literature sources that are relevant for the domain of software engineering, including journals and conferences, in order to guarantee the widest search range possible, while assuming quality of the considered publications. Based also on sources used in similar studies [8], we ended with the following digital libraries: **ACM Digital Library, IEEE Explore, ScienceDirect, Springer-Link, and Wiley Online Library**, libraries we queried for studies ranging from 2010 to 2024.

We chose 2010 as the starting point for our study for several reasons. Firstly, significant advancements in artificial intelligence, particularly in deep learning, occurred around that time. These advancements had a notable impact on software refactoring processes, as discussed in [15]. Secondly, beginning in 2010, there was a noticeable increase in the availability of data, largely due to the rise of open-source software

repositories. This trend is documented in [16]. Additionally, considering data from 2010 onward allows us to capture the emergence of new trends and developments in the field of AI-driven software refactoring.

To identify articles relevant to our chosen subject, we constructed a query comprising multiple keywords. A previous systematic literature review on software refactorings utilized a similar query structure [8]. We adopted the query presented in [8] and augmented it with additional keywords relevant to the field of artificial intelligence, and applied it to all five digital libraries. This modification aimed to refine the search results, focusing specifically on articles with ties to the artificial intelligence domain. The strings used in the query are presented in Table II.

TABLE II
KEYWORDS UTILIZED IN THE QUERY

Name	String	Domain
L1	software OR system OR code OR service OR diagram OR database OR architecture OR Model OR GUI OR user interface OR UI OR design OR artefact OR developer OR computer OR programming OR object-oriented OR implement OR mobile app OR cloud OR document	Software Systems
L2	refactor OR refactoring	Refactoring
L3	AI OR artificial intelligence OR ML OR machine learning OR intelligent algorithm OR deep learning OR intelligent method OR intelligent model	Artificial Intelligence

The resulting query is as follows: *(L1 AND L2 AND L3)*.

In order to address threats to validity of our approach and to complement the automatic results with findings from different studies, the next step was to perform snowballing, as described in [17]. Based on the first 3 pages results returned by Google Scholar for "refactoring" and "AI/ML" search (which we considered to be relevant enough for our purpose), we carried out a backward snowballing by inspecting the studies from the reference list (title, keywords and abstract). In this way, we added 29 new entries to our result set.

C. Inclusion and exclusion criteria

The following criteria must be satisfied in order to include primary studies in our selection:

- published between 2010 and 2024. We included papers until 31.03.2024;
- must be a research article
- published in a peer-reviewed venue (journal or conference proceeding);
- must use AI/ML methods for some refactoring stages.

We then excluded from our study: duplicates, primary studies not in English and primary studies classified as short papers or posters. Manual examination that included reading the abstract, and in some cases the entire paper, further excluded papers that did not address the scope. In addition,

given the relative symmetry of searching keywords (AI and refactoring), we identified in our SLR 15 primary studies addressing refactoring of AI/ML applications, which were also eliminated from this study. The complete selection process is illustrated in Figure 2.

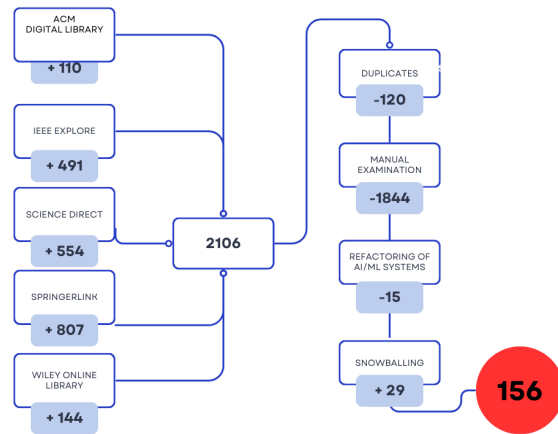


Fig. 2. Selection process for primary studies

D. Processing

Initial data extracted for each study included title, author, venue, publication date, and URL. This information facilitated manual examination and classification of the primary studies according to different dimensions:

- *research type*, as defined by [18], namely: validation research, evaluation research, solution proposal, philosophical paper, opinion paper, and experience paper;
- *refactoring life-cycle* as identified in [8]: detection, prioritization, recommendation, testing, documentation, prediction;
- *AI/ML method* to describe the method applied to refactoring.

III. RESULTS

A. Overview of results

Using the search strategy described in Section II-B, we successfully extracted a total of 1,986 articles from five different databases that met our initial criteria. Applying the manual review of titles and abstracts in order to extract the proper articles relating to AI/ML methods for refactoring, the count was reduced to 127. The application of snowballing techniques brought 29 additional articles that met the criteria. The final count of articles that were analyzed is 156.

Using Figure 3 as a representation, it can be observed that there has been a rising trend in the publication of articles focused on the use of artificial intelligence (AI) for refactoring starting from year 2010. Although initially low, with occasional spikes such as 2013 suggesting an intensive research focus, the trend continued upward, with a clearly observed breakthrough in the number of articles starting with

2016. From that point on, the number of articles increased substantially each year (above 10 articles per year), with a significant growth in 2020. The peak publication year for articles on this topic was 2023. This increase indicates a greater interest and advancements in the application of AI methodologies within the domain of refactoring. The year 2024 shows fewer articles compared to 2023, but it is too early to call it a decrease in interest, as this year's publications are considered only for the first three months.

The analysis of articles by research type presents a striking dominance of *solution*-oriented works (72.44%), greatly surpassing other categories such as *evaluation* (20.51%), *experience* (0.64%), and *opinion* (3.21%). *Solution* articles, which propose practical applications or answer specific research questions, outnumber *validation* (1.92%) articles, which test the applicability of existing theories or methods, by a substantial margin. There is a significant difference between *solution* and *evaluation* articles, albeit not as big as in the aforementioned case, highlighting a research landscape that is more focused on the innovation of solutions rather than the rigorous evaluation of their effectiveness. The presence of evaluation-focused research, though not as prevalent, is essential as it provides the critical assessments and validations that underpin the practical advancements represented by the *solution* articles. Articles with a *philosophical* (1.28%) angle, discussing the theoretical implications of AI in refactoring, appear to be rare. This distribution underscores a strong preference within the research community for contributions that offer tangible methodologies or innovations in the field of AI and refactoring. The distribution can be observed in Figure 4.

The resource package consisting of the articles dataset and the complete list of references can be found at the following URL: <https://doi.org/10.6084/m9.figshare.25710105.v1>.

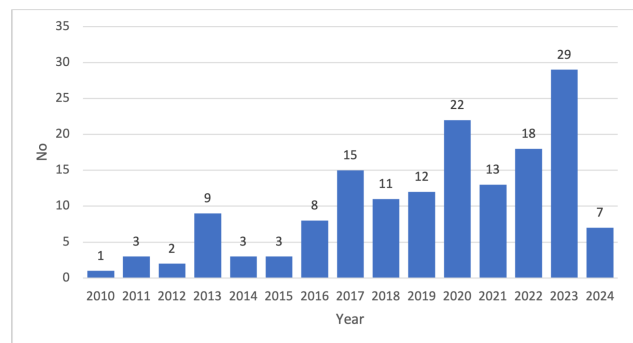


Fig. 3. Distribution of primary studies per years

B. Addressing refactoring life-cycle

Answer to RQ1: We based our investigation on the stages determined in [8], which represents a set of stages covering refactoring life-cycle in a 30 years SLR study. We summarized our result in the second column of Table III, followed by the staging results from [8], and the number of papers for each

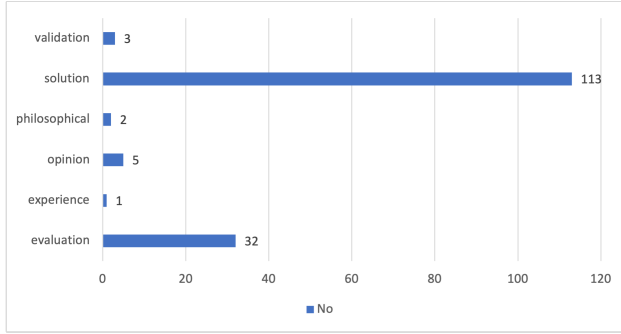


Fig. 4. Distribution of primary studies per research type

stage in 2023 identified in our SLR, as the last year exhibits a significant increase in topic under investigation (see Figure 3).

TABLE III
REFACTORING LIFE-CYCLE

Stage	%	% in 30 years [8]	2023 trend
Detection	37.18	28.65	10
Prioritization	8.33	9.43	2
Recommendation	32.69	16.18	7
Testing	1.28	18.44	1
Documentation	9.62	5.22	4
Prediction	10.90	4.18	5

As Table III shows, the majority of contributions address detection of refactoring opportunities (37.18% of papers) based on features like code smells, code metrics, or clustering, with most of the studies offering solutions (approx. 71%). Refactoring recommendations (32.69% of papers) also represent a topic of interest within the research community, with contributions being solution-(44 papers) and evaluation-oriented (7 papers). We notice that documentation, prioritization and testing are under-explored areas of research.

The particularity of applying AI/ML methods compared with all contributions to the refactoring life-cycle, as presented in [8], shows some similarities and differences: i) refactoring detection represents the stage with most contributions in both cases; ii) recommendation and prediction stages are benefiting from AI/ML solutions, while testing might not be a suitable problem to be solved with such methods. iii) We also consider that prioritization might better benefit from AI/ML solutions.

As 2023 experienced a significant increase in studies using AI/ML methods addressing refactoring life-cycle, we look at the general distribution of interest: detection, recommendation and prediction seem to keep attracting the same interest, with an increase in documentation of refactoring and a slight decrease in prioritization.

C. AI/ML methods in refactoring

Answer to RQ2: Various artificial intelligence (AI) methodologies were applied across all stages of software refactoring processes in the examined studies. These methodologies encompass:

- Natural Language Processing techniques such as BERT, TextRank (TR), Yake (Y), Fast Text, and others.
- Graph Neural Networks including GCN and GNN.
- Genetic Algorithms and Evolutionary Algorithms.
- Clustering.
- Regression analysis.
- Rule Mining.
- Constraint Satisfaction Problems.
- Multi-Objective Optimization approaches.
- Classification and Regression Trees.
- Ensemble Learning methods such as AdaBoost, Random Forest, Extra-trees, Gradient Boosting, and similar techniques.
- Deep Learning methodologies including MLP, DNN, LSTM, CNN, RNN, among others.
- Bayesian Methods.
- Support Vector Machines.

Further elaboration on this matter can be found in the resources accessible via the provided link <https://doi.org/10.6084/m9.figshare.25710105.v1>.

To categorize the AI methods used according to their learning type, we initially reviewed the abstracts. If the abstracts lacked sufficient information, we proceeded to analyze the full articles, particularly those available for free, to ascertain the learning types utilized by the AI methods. From the 156 articles included in our final dataset, 7 articles either lacked adequate details about the learning algorithms employed or were inaccessible in their entirety, with their abstracts being insufficiently clear on this matter. The distribution of the AI methods utilized is illustrated in Table IV.

TABLE IV
DISTRIBUTION OF EMPLOYED AI METHODS

Type of AI method	Number of articles
Supervised	88
Semi-Supervised	15
Unsupervised	37
Hybrid	9
Unspecified	7

The majority of the articles, specifically 56.41%, employed a supervised learning approach. This method entails the intelligent models learning from labeled data. Such an approach affords significant control over the model, allowing for the adjustment of training and validation datasets. This includes tasks such as balancing underrepresented classes and enhancing dataset diversity to enhance the model's learning capacity. The widespread use of supervised AI methods implies that researchers place emphasis on refining models and enhancing performance through the utilization of labeled data. This inclination may arise from a quest for heightened precision and dependability in addressing software refactorings, particularly within contexts of critical software development.

The utilization of unsupervised approaches, 23.71% out of the total, suggests that researchers in the field acknowledge the potential efficacy of unsupervised learning, especially in situations where obtaining labeled data proves challenging or

unfeasible. Unsupervised learning stands out for its capacity to uncover concealed patterns and structures within data without reliance on explicit labels. This attribute proves particularly advantageous when dealing with software refactorings, whose characteristics may lack full understanding or definition. Moreover, the integration of unsupervised AI methods signifies a proactive exploration of alternative avenues beyond conventional supervised learning.

Semi-supervised learning bridges supervised and unsupervised methods, blending labeled and unlabeled data in training. Its moderate adoption, namely 9.61% from the total, signals recognition of leveraging both data types' benefits, maximizing efficiency, especially in scenarios with limited labeled data. Employing semi-supervised learning techniques reflects a strategic endeavor to optimize model training efficiency, particularly in scenarios where obtaining labeled data may pose challenges in terms of availability or expense. This strategic use aims to enhance task accuracy and robustness in software refactoring tasks.

The use of hybrid AI methods, combining multiple learning types, reflects researchers' recognition of the benefits of integrating diverse techniques to address the complexities of software refactoring tasks. This approach allows for flexibility in leveraging different learning strengths, enabling tailored models to tackle specific challenges. Adopting hybrid AI methods signals an active exploration beyond conventional single-learning approaches, aiming to capitalize on the complementary strengths of supervised, semi-supervised, and unsupervised learning. Ultimately, researchers seek to enhance model accuracy, reliability, and adaptability.

After analyzing the AI methods employed by the considered studies, we extracted a list of 56 methods that were used, list that is available in the resource package from <https://doi.org/10.6084/m9.figshare.25710105.v1>. We have identified a selection of the five most frequently utilized AI methods from the pool of 56 intelligent algorithms employed. This approach allows us to discern common trends. Moreover, analyzing the top five algorithms enables us to gain insights into the predominant methodologies favored by developers and their potential impact on software refactoring stages.

TABLE V
TOP 5 MOST USED AI METHODS

AI method	Number of studies
Random Forests	18
Genetic Algorithm	17
Support Vector Machine	15
Convolutional Neural Network	12
Decision Trees	11

The prevalence of ensemble methods is evident as Random Forest and Genetic Algorithm (GA) emerge as the top choices, showcasing a clear inclination towards optimization techniques. This trend underscores developers' appreciation for ensemble methods' robustness and their capability to enhance performance. Additionally, the frequent utilization of Support Vector Machine (SVM) underscores its versatility

across a spectrum of software refactoring tasks, owing to its proficiency in classification and regression tasks. The prominence of Convolutional Neural Networks (CNN) underlines the growing significance of deep learning methodologies in respective studies, particularly in handling structured data like code snippets. CNNs' adeptness in tasks requiring pattern recognition or feature extraction is particularly noteworthy. Furthermore, the substantial usage of Decision Tree (DT) algorithms emphasizes their simplicity, interpretability, and effectiveness in classification and regression tasks. Often serving as foundational components in ensemble methods like Random Forests, DTs' frequent appearance alongside Random Forests elucidates their importance in the software refactoring toolkit.

As observed in Section III-B, the majority of the studies focused their investigations on the detection, recommendation, and prediction of software refactorings. An important observation can be made regarding the specific AI methods employed within each of the most scrutinized refactoring lifecycles. This insight sheds light on the nuances inherent in each lifecycle and the prevailing trends concerning the utilization of distinct AI algorithms across various scenarios.

The dominant AI methodology employed in the detection of software refactoring processes is represented by the Convolutional Neural Network, accounting for 20.68% of instances. This suggests the efficacy of CNNs in handling complex and varied data structures commonly found in codebases. This prevalence underscores the importance of advanced machine learning techniques in automating the identification of refactorings, potentially leading to increased efficiency and accuracy in software maintenance.

The predominant AI methodology utilized in recommending software refactoring processes is exemplified by the Genetic Algorithm, employed in 15.68% of instances. The widespread use of Genetic Algorithms in recommending software refactoring processes indicates their effectiveness in navigating complex optimization spaces inherent in software engineering tasks. This prevalence suggests a growing reliance on evolutionary computation techniques to automate and streamline the decision-making process in software maintenance and evolution. Additionally, it highlights the significance of using flexible and practical methods to tackle the changing and varied issues developers encounter when handling complex software and enhancing code quality.

As for the prediction of software refactoring, Random Forests represents the method utilized most frequently, accounting for 29.41% of cases. The extensive utilization of this intelligent algorithm in predicting software refactoring processes underscores their effectiveness in handling diverse and complex data patterns inherent in software development. This prevalence suggests a recognition of Random Forests' ability to provide reliable predictions, potentially leading to more informed decision-making in software maintenance and evolution. Furthermore, it highlights the growing importance of ensemble learning techniques in addressing predictive modeling challenges within the software engineering domain.

It's noteworthy that all AI methods used in the most studied

refactoring lifecycles are among the top 5 most utilized across all the studies we reviewed on this systematic literature review, as presented in Table V. This underscores their widespread applicability and effectiveness in addressing software refactoring challenges, reflecting a consistent preference among researchers for these robust methodologies. Another remark refers to the fact that evaluation studies included in our SLR involves more than one AI/ML method.

IV. DISCUSSION AND IMPLICATIONS

The intent of our literature review was to identify which AI/ML solutions are targeting different refactoring stages, but also to point out under-addressed refactoring stages and research types. A diverse set of contributions to a topic is beneficial for the research community but also has the potential to lead to impact in the business practices.

Our investigation confirms that the interest surrounding the application of AI/ML solutions to different problems is also reflected in refactoring research. A spike, mainly due to search based software engineering interest in 2013, was accompanied by a continuous interest starting from 2016, with a significant increase in 2023. The important aspect correlated with this increasing interest is that the overwhelming contributions offer solutions, with very few concentrating on evaluation, validation or experience. More scientific contributions of these types will allow their implementation in practice and a more realistic assessment in terms of benefits and costs.

From our expertise, stages like prediction and prioritization can better benefit from AI/ML solutions and can bring significant improvements to the refactoring process. In general, software systems predict the moment when to apply refactoring and anticipate the code fragments that may benefit from refactoring together with better deciding which part to refactor or how (prioritization) can have an important impact on the overall development costs and performance of the system [1].

Key findings:

- More evaluation, validation and experience research is required in order to make AI/ML solutions to refactoring stages implemented in software practices;
- There exist significant gaps between AI/ML solutions offered for different refactoring stages; refactoring prediction and prioritization can be better addressed by ML techniques;
- The majority of articles (56.41%) used supervised learning, highlighting a focus on refining models with labeled data for better precision in predicting software refactorings.
- Key AI methods like Random Forests and Genetic Algorithms emphasized optimization, while SVM, CNNs, and Decision Trees showcased versatility and effectiveness.

It is our strong belief that the results of this SLR offer substantial benefits to the research community through the comprehensive analysis of the AI/ML methods grouped for different refactoring stages, with our provided resource package allowing extensive filtering of papers. It gives op-

portunities for researchers to identify understudied topics or approaches.

The practitioners community can also benefit from this analysis, as automated or semi-automated tool support for refactoring still requires substantial effort and can integrate validated AI/ML methods.

V. RELATED WORK

Software refactoring has demonstrated its importance within the life-cycle of software products. As software products grow increasingly complex, there is a growing interest in their maintainability. Numerous studies have been conducted to analyze various software refactoring strategies and their impacts on different software products.

The work shown in [8] presents a systematic literature review on software refactoring and also proposes a platform that enables an easier categorization of software refactorings. The analysis encompasses scholarly articles addressing software refactoring, spanning the years 1990 to 2020. The synthesis covers the following themes: artifacts, refactoring tools, different approaches, and performance evaluation in refactoring research. The study provides a thorough examination of refactoring, classifying 3,183 publications according to several research dimensions. With 33.08% of studies focusing on testing to prevent problems, it highlights a large reliance on automation and indicates an organized refactoring life-cycle with steps for detection, prioritization, and implementation. Although code is the item that gets refactored the most, the report also discusses less prevalent domains such as database and GUI reworking. Over 34% of the refactoring objectives are dominated by object-oriented architectures, whereas cloud computing and mobile applications receive less attention. Most refactoring research uses search-based methods, which are highly regarded for their effectiveness in complicated situations. It also strongly recommends increased industry cooperation in the assessment of refactoring techniques.

In [19], a systematic literature review on automatic software refactoring processes (1996-2015) is presented, focusing on key areas: target, mechanism, strategy, purpose, and objectives. It highlights the use of cloning techniques, machine learning, and search-based algorithms to enhance coupling and cohesion, streamlining code architecture. Despite widespread adoption, only 29% of articles provide practical tool support, emphasizing the need for increased automation in identifying and rectifying refactoring issues. The study urges deeper investigation into high-level techniques aligned with industry standards, advocating for exploration at the software development model level to reduce future maintenance costs. Additionally, it underscores the overlooked aspect of behavior preservation during refactoring, signaling a call for comprehensive research and methodological refinement.

The study presented in [9] offers a SLR that examined empirical research on the impact of code restructuring on object-oriented software quality up until 2015. Of these, 76 papers were found to be relevant. It was found that some refactoring scenarios—such as Move Method and Extract Class—received

more attention than others, mostly affecting internal quality criteria like cohesion and coupling. Statistical methods were, however, rarely used to evaluate the importance of quality changes during refactoring. The results indicate that different refactoring situations have varying effects on quality; some circumstances strengthen coupling while increasing cohesion; therefore, it is advised not to apply generalizations without evaluating each case separately.

Another significant study is detailed in [5], which presents a mapping study categorizing machine learning (ML) techniques and tools utilized to address software engineering (SE) domain issues. The authors conducted an analysis of 177 papers sourced from 8 electronic databases, along with mapping the 86 most frequently addressed SE domain issues using ML. The study identified over 300 ML techniques across these papers, with only ten being present in more than 10% of them: Naive Bayes, Random Forest, Support Vector Machine, K-Nearest Neighbors, C4.5, Decision Trees, Artificial Neural Networks, Logistic Regression, Evolutionary Algorithms and Bayesian Networks. Notably, Naive Bayes emerged as the most commonly employed algorithm. Additionally, the study identified 35 ML tools utilized in the investigated research, with WEKA being the most prevalent. Furthermore, the authors employed ML techniques to cluster papers and identify the most similar ones in terms of content. This clustering revealed two hierarchical clusters, with 69% focused on prediction tasks and 31% on classification tasks.

The article referenced as [20] presents findings from a systematic literature review focused on Search-Based Refactoring (SBR), referring to the process of finding the best sequence of software refactoring processes to be applied in a software artifact. This review encompassed an analysis of 71 primary studies conducted between 2000 and 2006. The results indicate that code represents the most frequently addressed artifact, with evolutionary algorithms emerging as the predominant search technique. Additionally, it was observed that the generated solutions often constitute sequences of refactorings, typically drawn from Fowler's Catalog [1]. The primary metrics employed to assess these solutions include the count of detected bad smells, the number of changes, and a range of metrics specific to object-oriented artifacts. Experimentation typically involves the utilization of open-source programs and entails comparative evaluations with alternative approaches, with emphasis placed on analyses of execution time and fitness value.

This study stands out from prior Systematic Literature Reviews (SLRs) for several reasons. Primarily, it concentrates on investigating the integration of AI techniques within the phases of software refactoring, diverging from other studies that explore various aspects of software refactoring. Additionally, the scope of this study is narrowed specifically to analyzing each life cycle of the software refactoring processes, rather than encompassing the entire process. Furthermore, this study's focus lies solely on the application of AI methods within software refactoring processes, distinct from a broader examination of the entire software engineering domain, as

outlined in [5].

VI. THREATS TO VALIDITY

In this section, we discuss potential limitations and biases in our systematic literature review according to [21], which may impact the interpretation and generalizability of the findings. By acknowledging these threats, we aim to enhance the transparency and robustness of our conclusions.

Study Selection Validity: To mitigate the threat of replicability and repeatability of data, we established a clear search protocol in line with the standards set forth by [12]. Our automated search was conducted across five digital library portals to minimize study selection bias. Furthermore, the definition of search keywords was iterative and consistently vetted against similar studies ([8], [9]) to ensure thoroughness. In addition to our automated search, we handled duplicates studies and integrated an additional step—snowballing—to capture studies that could have been missed, thereby strengthening the construct validity of our review.

Data Validity: To assure data reliability, each article was subject to a manual verification by all three authors. The results were subsequently cross-verified, affirming the accurate application of the inclusion and exclusion criteria. While this approach limits publication bias, we acknowledge that our review's scope may omit pertinent studies published in other languages or in grey literature. This recognition is important for a comprehensive understanding of the study's boundaries.

Research Validity: We strived to enhance the generalizability of the study by formulating a search string that encompassed a diverse array of terms from both AI and Software Engineering fields, aiming to broaden the literature coverage. Threats related to the conclusions of this investigation were mitigated by continuous comparisons with similar studies [8], [9], [19]. This practice helps in underpinning the credibility of our findings and ensuring that the conclusions drawn are reflective of the broader research landscape within this domain.

VII. CONCLUSIONS

This paper constructed a systematic literature review (SLR) that provides a comprehensive examination of the application of AI technologies in the domain of software refactoring. We followed established methodologies [12] and a rigorous process of extracting and analyzing primary studies. Beginning with an initial set of 1,986 articles, the detailed review process refined this to 156 studies that illustrate the state-of-research and adoption of AI/ML in the field of software refactoring. The publication trends over the years 2010-2024 highlighted a significant increase in research interest and productivity, with a higher, constant rise from 2016 onward, publication peaking in 2022 and 2023. This suggests a resilient engagement within the research community towards leveraging AI/ML for enhancing refactoring processes.

We focused on key research questions that explored what lifecycle phases of refactoring are addressed by artificial intelligence methods and the specific methods used by researchers. We found that refactoring detection represents the

stage with the most contributions, followed by recommendation and prediction. We also consider that prioritization, together with prediction, can benefit more from ML solutions and bring improvements to software development processes, more notably on the overall development costs [1].

A noticeable interest in using hybrid AI approaches per article was observed, reflecting researchers' recognition of integrating multiple techniques in order to improve model accuracy, reliability, and adaptability.

Another key finding is the lower number of articles focusing on evaluation, validation and experience research. This highlights the importance of such research being addressed in order to better adopt AI/ML solutions to refactoring scenarios in real-life software environments. The most commonly used ML methods are Random Forests, Genetic Algorithms, SVM, CNNs and Decision Trees. This highlights their efficacy and versatility when it comes to applying them to the refactoring process. Furthermore, various different methods were applied on different lifecycle stages, implying some methods work better than others when addressing particular challenges in refactoring. Most of the models (56.41%) were refined using supervised learning, followed by unsupervised learning and semi-supervised learning, highlighting the fact that data labeling may offer better precision in predicting software refactorings.

Future work includes a more detailed investigation of primary studies, search for model performance statistics, collect evaluation methods used by the researchers, and create an overview on the state-of-the-art of the solutions proposed for studying each refactoring life cycle. A more comprehensive mapping study can be constructed based on this SLR, by classifying the AI methods on whether they belong to Deep Learning, Machine Learning or other AI fields, in order to better see the trend for choosing the best method when it comes to software refactoring research. In the near future, we expect to experience an increased interest in applying LLM solutions to refactoring, which can benefit, as this study, by a systematic mapping.

REFERENCES

- [1] M. Fowler, *Refactoring: Improving the Design of Existing Code*. Boston, MA, USA: Addison-Wesley, 1999.
- [2] O. Chaparro, G. Bavota, A. Marcus, and M. D. Penta, "On the impact of refactoring operations on code quality metrics," in *Proceedings of the 2014 IEEE International Conference on Software Maintenance and Evolution*, ser. ICSME '14. USA: IEEE Computer Society, 2014, p. 456–460. [Online]. Available: <https://doi.org/10.1109/ICSME.2014.73>
- [3] L. Traini, D. Di Pompeo, M. Tucci, B. Lin, S. Scalabrino, G. Bavota, M. Lanza, R. Oliveto, and V. Cortellessa, "How software refactoring impacts execution time," *ACM Trans. Softw. Eng. Methodol.*, vol. 31, no. 2, 2021. [Online]. Available: <https://doi.org/10.1145/3485136>
- [4] R. Verdecchia, R. A. Saez, G. Procaccianti, and P. Lago, "Empirical evaluation of the energy impact of refactoring code smells," in *ICT4S2018. 5th International Conference on Information and Communication Technology for Sustainability*, ser. EPIc Series in Computing, B. Penzenstadler, S. Easterbrook, C. Venters, and S. I. Ahmed, Eds., vol. 52. EasyChair, 2018, pp. 365–383. [Online]. Available: <https://easychair.org/publications/paper/MxpT>
- [5] O. Borges, J. Couto, D. Ruiz, and R. Prikladnicki, "How machine learning has been applied in software engineering?" in *Proceedings of the 22nd International Conference on Enterprise Information Systems - Volume 2: ICEIS*, INSTICC. SciTePress, 2020, pp. 306–313.
- [6] University of Waikato, "Weka - data mining software in java," <https://sourceforge.net/projects/weka/>.
- [7] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay, "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- [8] C. Abid, V. Alizadeh, M. Kessentini, T. do Nascimento Ferreira, and D. Dig, "30 years of software refactoring research: A systematic literature review," *CoRR*, vol. abs/2007.02194, 2020. [Online]. Available: <https://arxiv.org/abs/2007.02194>
- [9] J. Al Dallal and A. Abdin, "Empirical evaluation of the impact of object-oriented code refactoring on quality attributes: A systematic literature review," *IEEE Transactions on Software Engineering*, vol. 44, no. 1, pp. 44–69, 2018.
- [10] P. Ralph and S. Baltes, "Paving the way for mature secondary research: the seven types of literature review," in *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, ser. ESEC/FSE 2022. Association for Computing Machinery, 2022, p. 1632–1636.
- [11] Paul Ralph (ed.), "Acm sigsoft empirical standards for software engineering research, version 0.2.0," 2021. [Online]. Available: <https://github.com/acmsigsoft/EmpiricalStandards>
- [12] B. A. Kitchenham and S. Charters, "Guidelines for performing systematic literature reviews in software engineering," Keele University and Durham University Joint Report, Tech. Rep. EBSE 2007-001, 07 2007. [Online]. Available: https://www.elsevier.com/_data/promis_misc/525444systematicreviewsguide.pdf
- [13] D. S. Cruzes and T. Dybå, "Synthesizing evidence in software engineering research," in *Proceedings of the 2010 ACM-IEEE International Symposium on Empirical Software Engineering and Measurement*, ser. ESEM '10. Association for Computing Machinery, 2010.
- [14] D. Binkley, M. Ceccato, M. Harman, F. Ricca, and P. Tonella, "Automated refactoring of object oriented code into aspects," in *21st IEEE International Conference on Software Maintenance (ICSM'05)*, 2005, pp. 27–36.
- [15] C. Watson, N. Cooper, D. N. Palacio, K. Moran, and D. Poshvanyk, "A systematic literature review on the use of deep learning in software engineering research," vol. 31, no. 2, mar 2022. [Online]. Available: <https://doi.org/10.1145/3485275>
- [16] W. Jung, E. Lee, and C. Wu, "A survey on mining software repositories," *IEICE TRANSACTIONS on Information and Systems*, vol. E95-D, no. 5, pp. 1384–1406, 05 2012.
- [17] C. Wohlin, "Guidelines for snowballing in systematic literature studies and a replication in software engineering," in *Proceedings of the 18th International Conference on Evaluation and Assessment in Software Engineering*, ser. EASE '14. New York, NY, USA: Association for Computing Machinery, 2014. [Online]. Available: <https://doi.org/10.1145/2601248.2601268>
- [18] K. Petersen, R. Feldt, S. Mujtaba, and M. Mattsson, "Systematic mapping studies in software engineering," in *Proceedings of the 12th International Conference on Evaluation and Assessment in Software Engineering*, ser. EASE'08. BCS Learning & Development Ltd., 2008, p. 68–77.
- [19] A. A. B. Baqaïs and M. Alshayeb, "Automatic software refactoring: a systematic literature review," *Software Quality Journal*, vol. 28, no. 2, p. 459–502, Jun. 2020.
- [20] T. Mariani and S. R. Vergilio, "A systematic review on search-based refactoring," *Information and Software Technology*, vol. 83, pp. 14–34, 2017. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0950584916303779>
- [21] A. Ampatzoglou, S. Bibi, P. Avgeriou, M. Verbeek, and A. Chatzigeorgiou, "Identifying, categorizing and mitigating threats to validity in software engineering secondary studies," *Information and Software Technology*, vol. 106, 02 2019.